
Pie-Lab 2025年暑期培训

李秋予

Pie-Lab

北京理工大学计算机学院
北京市海淀区中关村南大街
786362411@qq.com

Abstract

以下是对CIFAR-10图像使用注意力机制分类任务的实验报告，包括Attention的数学原理简述、模型结构设计、实现细节与关键代码说明、与CNN模型对比的实验设置与结果分析以及总结与反思。

1 数学原理

注意力机制(Attention Mechanism)是机器学习中的一种数据处理方法，广泛应用在自然语言处理、图像识别以及语音识别等各种不同类型的机器学习任务中。注意力机制对不同信息的关注程度（重要程度）由权值来体现，注意力机制可以视为查询矩阵(Query)、键(key)以及加权平均值构成了多层感知机(Multilayer Perceptron, MLP)。

注意力是将一个查询（Query）和键值对（Key, Value）映射到输出的方法。公式中的Q, K, V均为矩阵，通过计算Q和K的相似性或者相关性，得到每个K对应V的权重系数，然后对V进行加权求和，即得到最终的Attention数值。所以，本质上Attention机制是V进行加权求和，而Q和K用来计算对应V的权重系数。

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

1.1 Query、Key、Value 的含义和作用

1. Query（查询）

查询向量用于确定当前处理位置需要关注的信息焦点，例如在自然语言处理中，Query通常对应需要解析的词语（如代词“他”的指代对象）。

2. Key（键）

键向量用于表示其他元素的特征标签，例如在句子中每个词的特征标签包含主语、动词、修饰语等属性，帮助确定Query的关联对象。

3. Value（值）

值向量对应实际需要提取的信息内容，例如在文本处理中，Value可能包含具体词汇的语义或上下文关联信息。

- Key和Value矩阵来自encoder的输入 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m$.
- Query矩阵来自decode的输入 $\mathbf{x}'_1, \mathbf{x}'_2, \dots, \mathbf{x}'_t$.
- Keys: $\mathbf{k}_i = \mathbf{W}_K \mathbf{x}_i$
- Values: $\mathbf{v}_i = \mathbf{W}_V \mathbf{x}_i$

- Query: $q_j = W_Q x'_j$

本质是用“问题线索”（Query）在信息（Key+Value）中定位最相关的内容，通过“匹配度打分”（权重）让模型把精力集中在关键信息上，忽略无关信息。就像找东西时，会下意识把目光聚焦在最可能的地方，注意力机制让计算机也能“智能聚焦”，提升信息处理效率。

1.2 Attention的数学原理

注意力的思想，类似于寻址。给定Target中的某个元素Query，通过计算Query和各个Key的相似性或相关性，得到每个Key对应Value的权重系数，然后对Value进行加权求和，即得到最终的Attention数值。所以，本质上Attention机制是Source中元素的Value值进行加权求和，而Query和Key用来计算对应Value的权重系数。

- Source：由一系列的<Key, Value>键值对构成。
- Query：给定的Target元素；
- Key：Source中元素的Key值；
- Value：Source中元素的Value值；
- 权重系数：Query与key的相似性或相关性；
- Attention Value：对Value值进行加权求和；

注意力机制计算过程可以细化为如下三个阶段：

第一阶段，计算 Query和不同 Key 的相关性，即计算不同 Value 值的权重系数；

第二阶段，对上一阶段的输出进行归一化处理，将数值的范围映射到 0 和 1 之间。

第三阶段，根据权重系数对Value进行加权求和，从而得到最终的注意力数值。

1.第一阶段

计算 Query和不同 Key 的相似性或者相关性，即计算不同 Value 值的权重系数。

$$Similarity(Query, Key_i) = Query * Key_i \quad (2)$$

2.第二阶段

对第一阶段得到的原始权重进行归一化处理，将数值的范围映射到 0 和 1 之间。

根据产生方法的不同，第一阶段产生的分值的取值范围也不一样，第二阶段引入类似SoftMax的计算方式对第一阶段的得分就行数值转换。一方面，可以进行归一化，将原始计算分值整理成所有元素权重之和为1的概率分布；另一方面，通过SoftMax的内在机制更加突出重要元素的权重。即一般采用如下公式：

$$a_i = Softmax(Sim_i) = \frac{e^{Sim_i}}{\sum_{j=1}^{L_x} e^{Sim_j}} \quad (3)$$

3.第三阶段

根据权重系数对Value进行加权求和，从而得到最终的注意力数值：

$$Attention(Query, Source) = \sum_{i=1}^{L_x} a_i * Value_i \quad (4)$$

1.3 Self-Attention的数学原理

Self-Attention的数学原理与Attention有一定相似之处

注意力机制中，Q来自外部，通过 W_q 计算得到，Q和K/V来自不同序列，K、V来自同一组输入，经过 W_k 、 W_v 得到，输出数量 r 由Q的数量决定，与输入 m 无关。

自注意力机制中，Q、K、V来自同一组输入，经过 W_k 、 W_q 、 W_v 得到，输出 c_1 到 c_m 与输入 x_1 到 x_m 一一对应。

Attention：像”老师（Decoder）根据问题（Q）从课本（Encoder）中找答案（K/V）”。（输出 c 是老师整理的回答，数量与问题相关）

Self-Attention：像”学生小组讨论，每个人（ x_i ）通过与其他成员交流更新自己的观点（ c_i ）”。（输出 c_m 是讨论后的新观点，数量与参与者相同）

1.4 模型结构设计

1.图像分块 (Patch Embedding)

图像分块的处理过程通过卷积操作实现。该模块将二维图像数据转换为适合Transformer处理的序列化表示，其具体实现包含以下几个步骤：

首先，输入图像经过一个 $\text{kernel-size}=4$ 且 $\text{stride}=4$ 的卷积层进行处理，该卷积核的尺寸与移动步长都与预设的patch大小一致。因此对于 $32*32$ 的输入图像，会产生 $8*8=64$ 个不重叠的图像块（ $8=32/4$ ）。此外，卷积层的输入通道数为3（对应RGB三通道），输出通道数设置为256，与Transformer的隐藏层维度保持一致。

输入数据首先以 $[\text{batch-size}, 3, 32, 32]$ 的四维张量形式进入网络。经过卷积操作后，输出变为 $[\text{batch-size}, 256, 8, 8]$ 的形式。每个patch（有rgb三维）被嵌入为256维度（每个 $4*4*3$ 的patch与256个核做点积，使得每个patch $\rightarrow$ 256维向量），为了适应Transformer的序列处理特性，这个四维张量需要被展平和重组为三维的序列格式。

展平和重组操作，首先使用 $\text{flatten}(2)$ 方法将最后两个空间维度展平，得到 $[\text{batch-size}, 256, 64]$ 的中间表示；然后通过 $\text{transpose}(1,2)$ 操作交换第二和第三维度，最终形成 $[\text{batch-size}, 64, 256]$ 的标准序列格式。在这个过程中，每个图像块被转换为一个256维的特征向量，64个这样的向量按空间顺序排列，构成了Transformer的输入序列。

2.位置编码

Class Token通过 $\text{nn.Parameter}(\text{torch.zeros}(1, 1, \text{embed-dim}))$ 初始化一个可学习的参数矩阵。这个 $1*1*256$ 的张量作为模型的类令牌，其维度与图像块嵌入保持相同（256维）。在前向传播时，令牌会通过 $\text{expand}(\text{batch-size}, -1, -1)$ 来广播到批处理大小。这个扩展后的类令牌与图像块序列，通过 $\text{torch.cat}([\text{class-token}, \text{patch-embeddings}], \text{dim}=1)$ 进行拼接，将类令牌插入到所有图像块之前，形成 $65*256$ 的序列（64个图像块+1个类令牌）。使得类令牌能够通过后续的Transformer层与所有图像块进行全局交互，最终这个令牌的状态将作为整个图像的特征用于分类任务。

位置编码的实现采用可学习的1D位置编码方案。初始化时创建 $1*65*256$ 的参数矩阵，其中65对应类令牌加上64个图像块的位置，256与嵌入维度一致。这个位置编码不是固定的人工设计模式（如正弦函数），而是通过 $\text{nn.init.trunc_normal}(\text{self.pos_embed}, \text{std}=0.02)$ 进行截断正态分布初始化，确保初始值在合理范围内。在前向传播中，位置编码会通过广播机制自动扩展到当前批处理大小，然后与嵌入序列进行逐元素相加： $x = x + \text{self.pos_embed}$ 。保留原始特征信息，又注入位置信息。

3.多头注意力机制

该模块首先将256维的嵌入向量分割为8个32维的子空间，每个子空间独立计算注意力权重。具体实现时，输入序列分别通过三个独立的线性层（ W_q 、 W_k 、 W_v ）投影到查询、键和值空间，形成Q、K、V三个矩阵。这些矩阵随后被重塑为 $[\text{batch-size}, \text{num-heads}, \text{seq-len}, \text{d-k}]$ 的四维张量，其中65个位置（含class token）的特征被分配到8个注意力头，每个头处理32维的特征子空间。注意力权重的计算遵循公式，通过对Q和K的转置矩阵做矩阵乘法后除以 $\sqrt{32}$ 进行数值稳定，再经过softmax归一化后与V矩阵相乘。最终，8个头的输出在特征维度拼接回256维，并通过最后的线性投影层 W_o 进行特征融合。

4.Transformer编码器

Transformer编码器的堆叠，每层包含两个核心子模块。第一个子模块是多头自注意力机制，输入特征首先经过LayerNorm归一化，然后进入注意力计算，计算结果与原始输入相加形成残差连接。第二个子模块是前馈神经网络，由两个线性变换层和GELU激活函数组成，同

样采用先归一化再计算最后残差连接的流程。为防止过拟合，每个子层后进行dropout操作(设置为0.3)，而层间残差连接则确保了梯度在深层网络中的有效传播。

5. 分类头

网络首先提取序列中第一个位置(class token)对应的256维向量，这个向量已经通过多层Transformer编码器聚合了全局图像信息。随后对该向量进行层归一化处理以稳定特征分布，最后通过一个简单的全连接层将256维特征映射到10维的分类空间(对应CIFAR-10的10个类别)。

2 对比实验及可视化

2.1 CNN模型与Resnet

在相同训练环境下，CNN最终准确率达77.46%，最佳准确率出现在第28个epoch；ResNet最终准确率为85.70%，准确率提升8.24%，最佳准确率出现在第29个epoch，残差连接在提升模型性能上有所提升。CNN总训练时间为790.63秒，ResNet总训练时间为780.80秒，CNN平均epoch时间为26.35秒，ResNet平均epoch时间为26.03秒。

```
==== 对比结果 ====  
CNN最终准确率: 77.46%  
ResNet最终准确率: 85.70%  
准确率提升: 8.24%  
  
==== 时间对比 ====  
CNN总训练时间: 790.63秒  
ResNet总训练时间: 780.80秒  
CNN平均epoch时间: 26.35秒  
ResNet平均epoch时间: 26.03秒  
CNN最佳准确率出现在第 28 个epoch  
ResNet最佳准确率出现在第 29 个epoch
```

Figure 1: CNN与Resnet结果对比

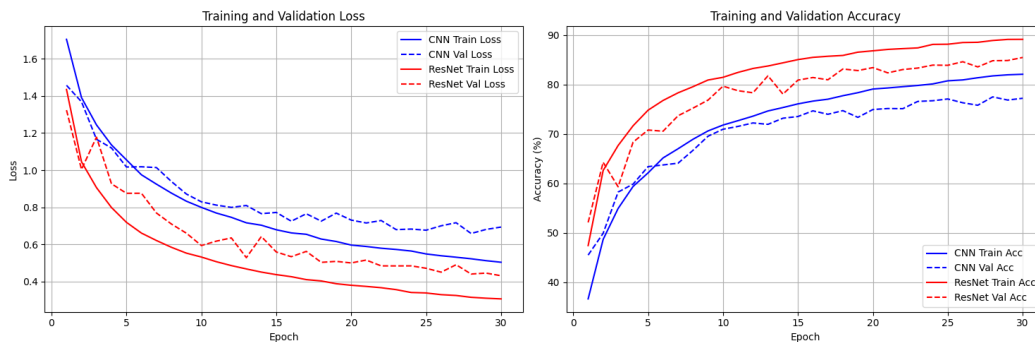


Figure 2: CNN与ResNet结果可视化

2.2 注意力机制

Attention模型以80.13%测试准确率介于CNN和ResNet之间，但Attention模型的训练/验证准确率曲线展现出更好的同步性。总训练时间为719.10秒，平均每个epoch训练时间为23.97秒。

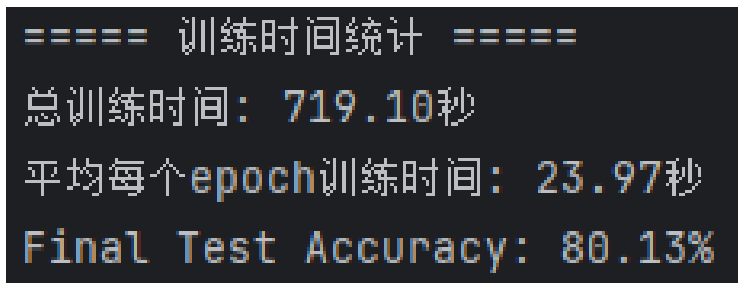


Figure 3: 注意力机制结果

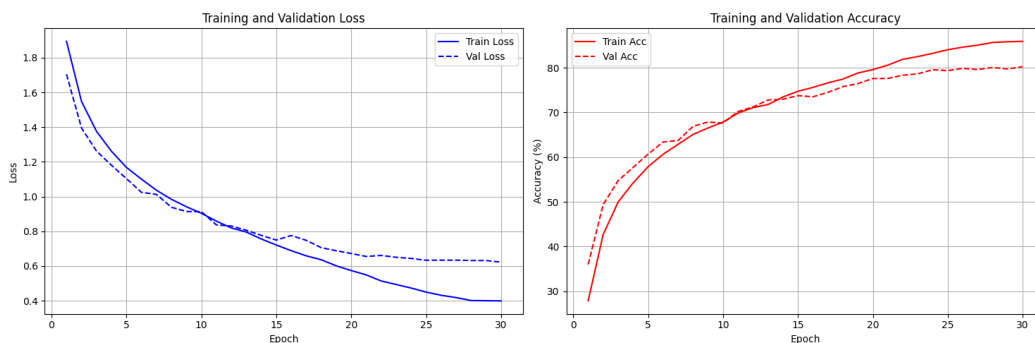


Figure 4: 注意力机制结果可视化

3 总结与分析

在CIFAR-10数据集上，ResNet模型展现出显著优势，其85.70%的最终准确率较CNN模型的77.46%提升了8.24个百分点。ResNet总训练时间为780.80秒，仅比CNN的790.63秒快1.2%，平均每个epoch的训练时间也基本持平（ResNet 26.03秒 vs CNN 26.35秒）。从收敛速度来看，两个模型都在训练后期达到最佳状态，CNN在第28个epoch、ResNet在第29个epoch取得最高准确率。

ViT模型的训练数据，虽然其80.13%的测试准确率介于CNN和ResNet之间，但在训练效率上具有明显优势。ViT的总训练时间仅为719.10秒，比CNN和ResNet分别节省了9%和7.9%的时间，平均每个epoch耗时23.97秒，展现出Transformer架构在并行计算方面的优势。这种效率优势使得ViT在需要快速迭代的场景中有更强的能力，特别是在更大规模数据集上可能展现出更大的潜力。综合来看，ResNet在准确率上保持领先，而ViT在训练效率上更胜一筹，CNN则作为基础模型提供了性能基准。实验结果表明，不同架构的神经网络在准确率和训练效率上存在明显的权衡关系，用户可根据具体应用场景的需求选择合适的模型架构。